

EOD – AWS Load Balancer Tasks

Date: 16 September 2026

Environment: AWS

Region: US East (N. Virginia)

Tasks Completed

1. Configure Classic Load Balancer (CLB)

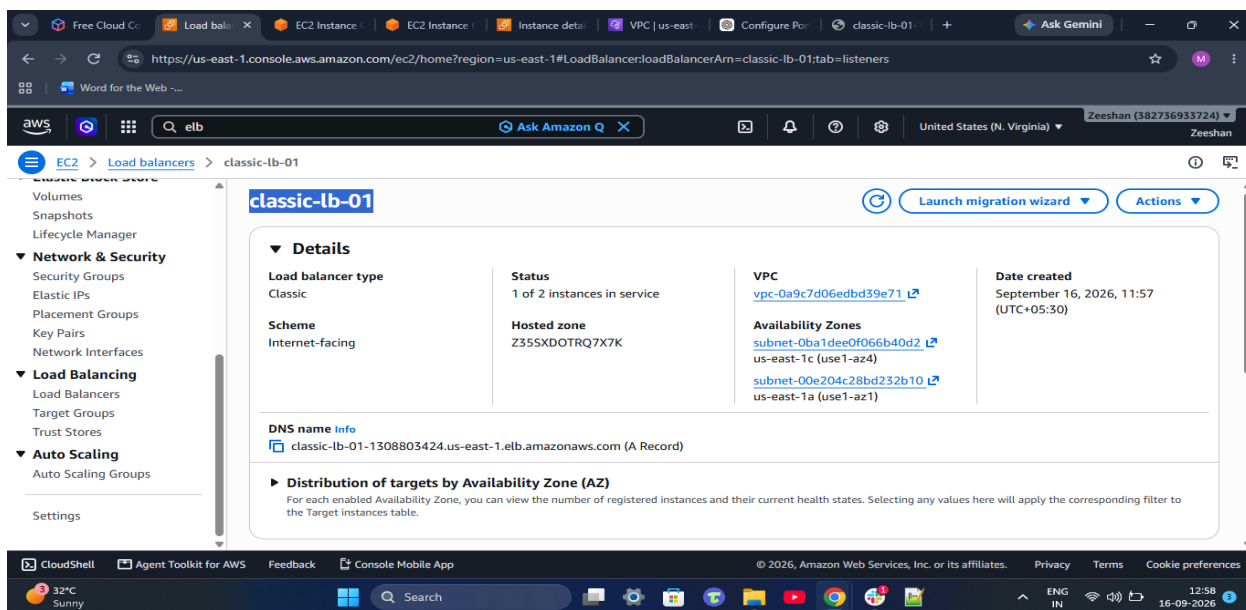
Objective: Configure a Classic Load Balancer to distribute application traffic to EC2 instances.

Steps:

1. Created a Classic Load Balancer named classic-lb-01.
2. Selected the required VPC and Availability Zones.
3. Registered EC2 instances with the Load Balancer.
4. Configured HTTP listeners.
5. Configured application ports according to the application setup.
6. Checked the health status of registered instances.
7. Tested the application using the CLB DNS name.

Result: Classic Load Balancer was configured and tested.

Evidence:



2. Configure Application Load Balancer (ALB)

Objective: Configure an Application Load Balancer for HTTP application traffic.

Steps:

1. Created an Application Load Balancer.
2. Selected **Internet-facing** scheme.
3. Selected the required VPC and subnets.
4. Configured the Security Group.
5. Created a Target Group.
6. Registered the required EC2 instance(s).
7. Configured the HTTP listener.

8. Added a listener rule/default action to forward traffic to the Target Group.
9. Verified target health.
10. Tested the application using the ALB DNS name.

Result: Application Load Balancer was configured successfully.

Evidence:

The screenshot displays the AWS Management Console interface. The top navigation bar shows the user is logged in as Zeeshan (382736933724) in the United States (N. Virginia) region. The main content area shows the configuration for a Target Group with the ARN `arn:aws:elasticloadbalancing:us-east-1:382736933724:targetgroup/httpd/86115d7b18649415`. The configuration includes:

- Target type:** Instance
- Protocol:** HTTP
- Port:** 80
- Protocol version:** HTTP1
- VPC:** [vpc-0a9c7d06edbd39e71](#)
- IP address type:** IPv4
- Load balancer:** [APP-LB-01](#)

The 'Targets' tab shows 1 total target, which is 'Healthy'. There are 0 Unhealthy, 0 Unused, 0 Initial, and 0 Draining targets. Below this, the 'Health checks' tab is selected, showing 'Distribution of targets by Availability Zone (AZ)'.

Below the console screenshot, a browser window is shown with the URL `http://app-lb-01-668527738.us-east-1.elb.amazonaws.com:82/sample/`. The page content reads:

welcome to my sample page

The screenshot shows a web browser window with the URL `http://app-lb-01-668527738.us-east-1.elb.amazonaws.com:82`. The page displays the 'Welcome to nginx!' message, indicating that nginx is successfully installed and working. The text on the page includes:

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

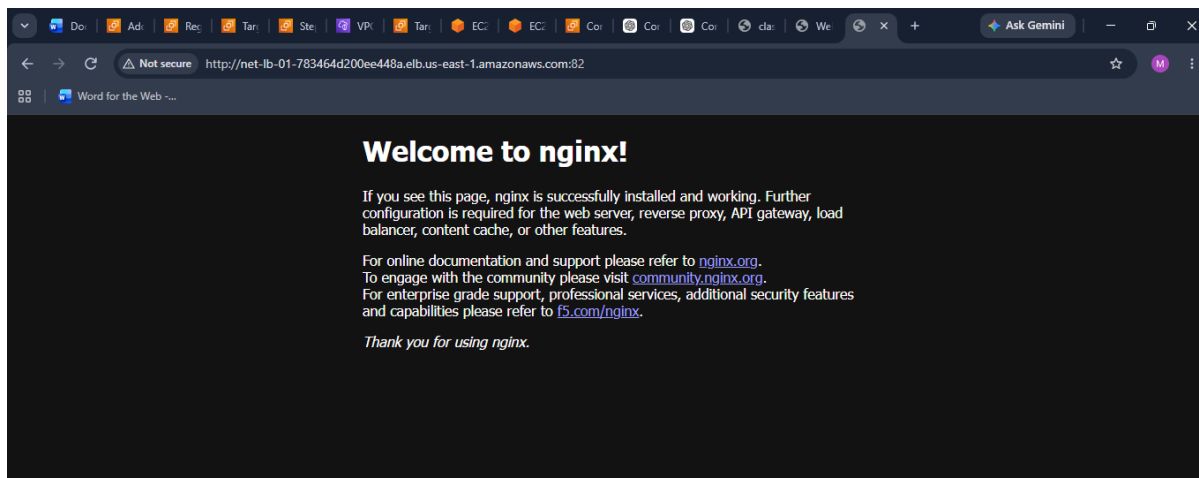
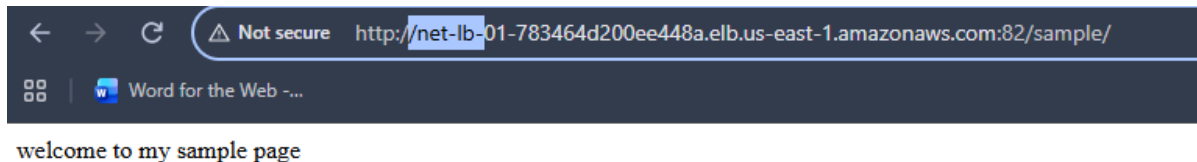
3. Configure Network Load Balancer (NLB)

Objective: Configure a Network Load Balancer for application traffic.

Steps:

1. Created a Network Load Balancer.
2. Selected the required VPC and Availability Zones.
3. Created a Target Group.
4. Registered the required EC2 instance(s).
5. Configured the required listener and port.
6. Verified target health.
7. Tested the application using the NLB DNS name.

Result: Network Load Balancer was configured and tested.



4. Attach SSL Certificate to Application Load Balancer

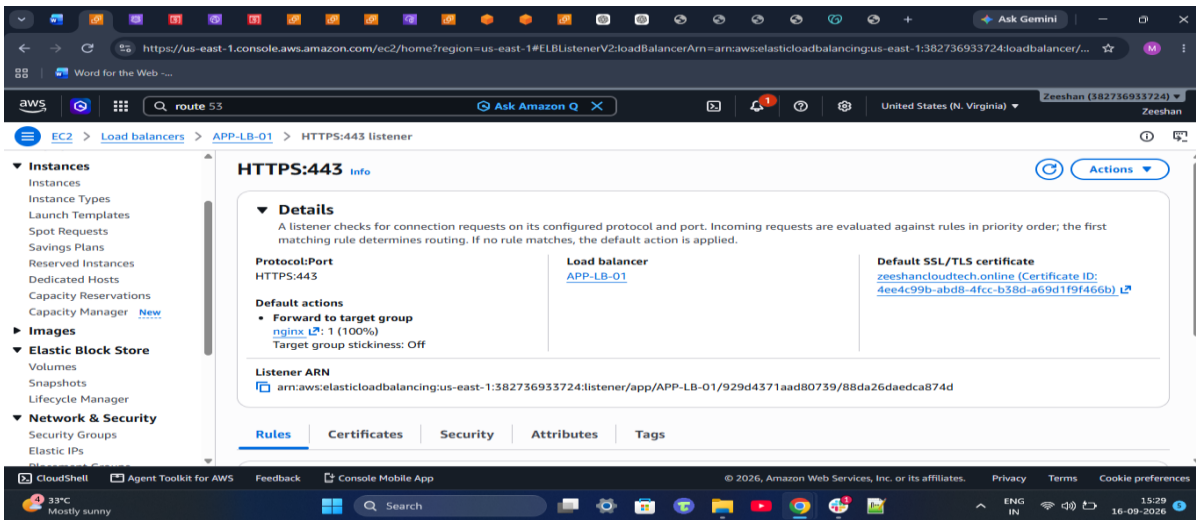
Objective: Enable HTTPS for the application using an SSL/TLS certificate.

Steps:

1. Opened **AWS Certificate Manager (ACM)**.
2. Selected the required SSL/TLS certificate.
3. Opened the ALB listener configuration.
4. Added an **HTTPS listener on port 443**.
5. Selected the ACM certificate.
6. Configured the HTTPS listener to forward traffic to the Target Group.
7. Tested the application using HTTPS.

Result: SSL/TLS certificate was attached to the Application Load Balancer and HTTPS was enabled.

Evidence:



5. Map Application Load Balancer to Route 53

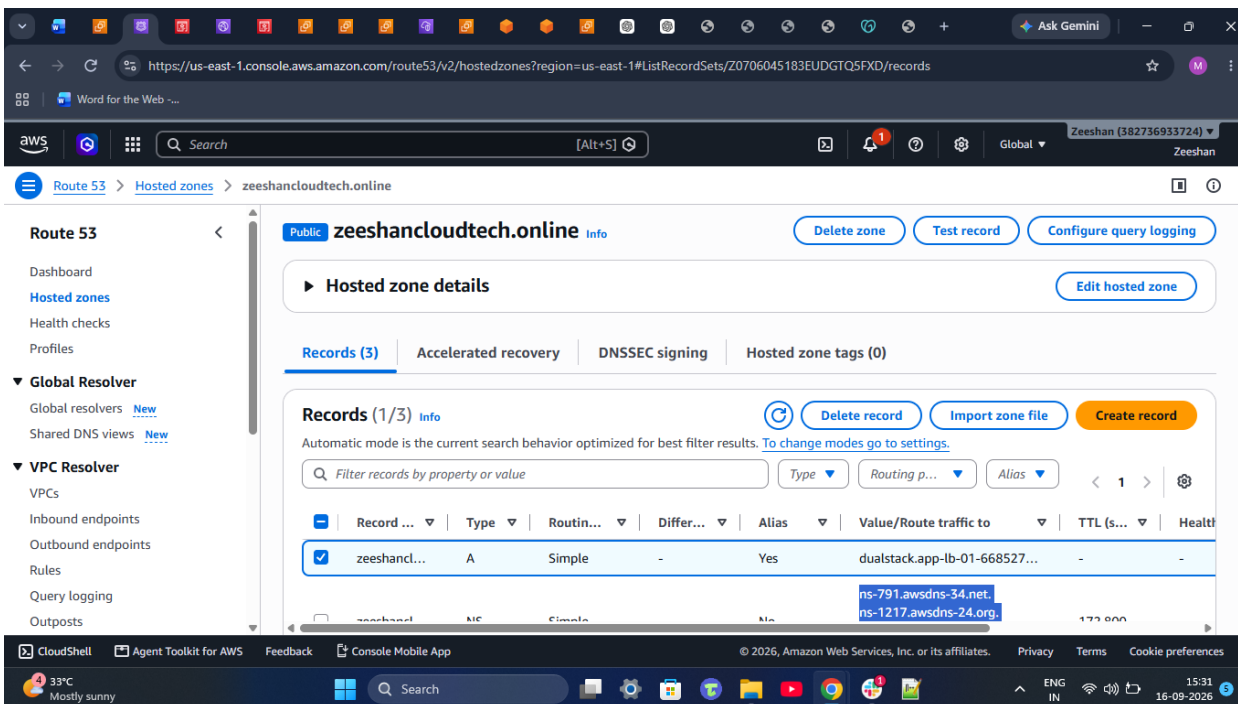
Objective: Map a custom domain name to the Application Load Balancer.

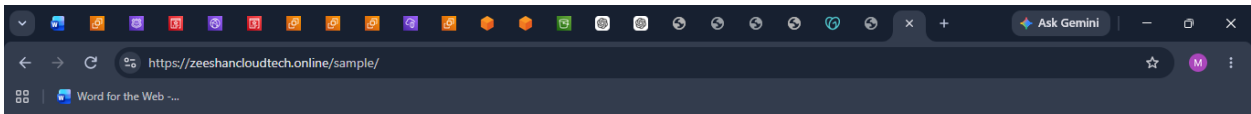
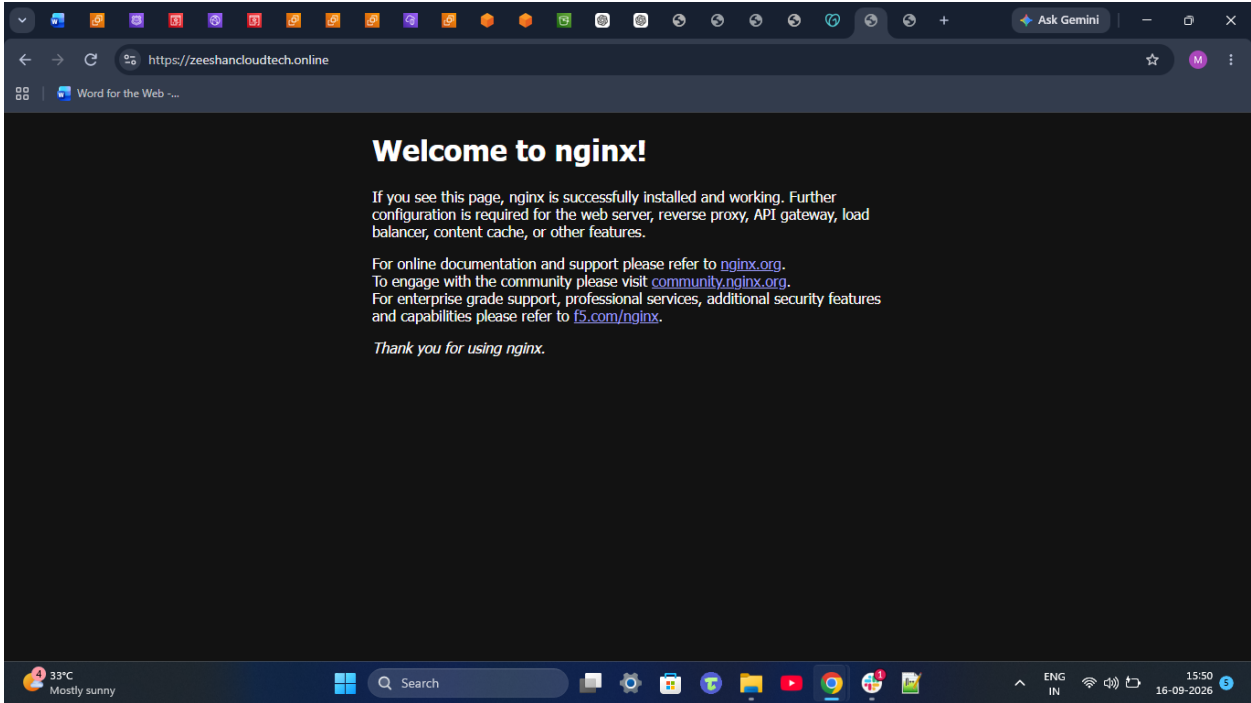
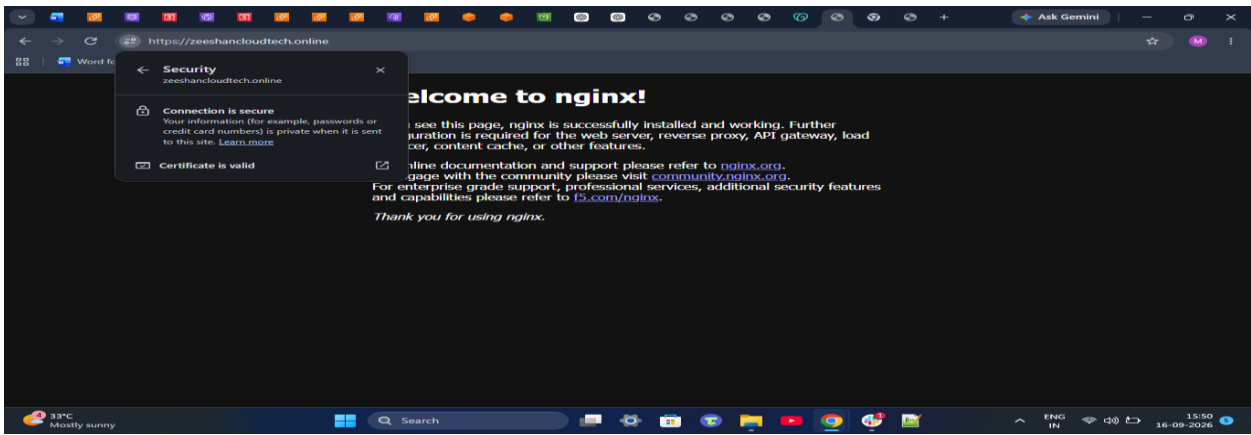
Steps:

1. Opened **Route 53**.
2. Selected the required Hosted Zone.
3. Created an **A record**.
4. Selected **Alias**.
5. Selected **Application Load Balancer** as the alias target.
6. Selected the required ALB.
7. Saved the record.
8. Tested the domain in the browser.

Result: The custom domain was successfully mapped to the Application Load Balancer.

Evidence:





welcome to my sample page



6. Push Application Load Balancer Logs to S3

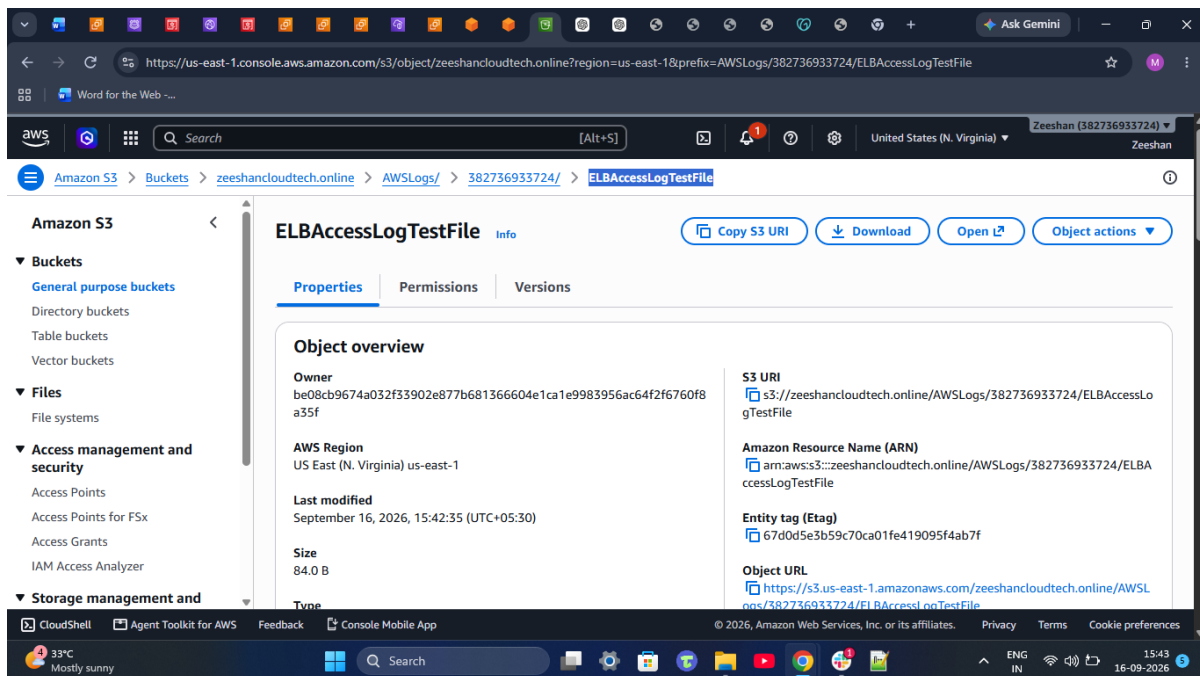
Objective: Store Application Load Balancer access logs in an S3 bucket.

Steps:

1. Created an S3 bucket for ALB logs.
2. Configured the required bucket permissions.
3. Opened the Application Load Balancer.
4. Enabled **Access Logs**.
5. Selected S3 as the storage location.
6. Entered the S3 bucket name.
7. Saved the ALB attributes.
8. Generated application traffic.
9. Checked the S3 bucket for ALB log files.

Result: Application Load Balancer access logs were configured to be stored in S3.

Evidence:



Conclusion

Successfully configured and tested **Classic Load Balancer, Application Load Balancer, and Network Load Balancer**. SSL/TLS was configured for the ALB, the ALB was mapped to Route 53, and ALB access logs were configured to be stored in Amazon S3.